NebulaStack

Prototipo ejecutable y consolidación arquitectónica nube-nativa



Equipo	Team B
Integrantes	Alejandra Mendoza Aguilera · Julián Ricardo Cipamoncha Rincón · Ciro Adolfo Castro Rangel · Fabio Álvarez
Asignatura	Cloud-Based Software Engineering (2026-II)
Institución	Universidad Nacional de Colombia — Departamento de Ingeniería de Sistemas e Industrial
Profesor	Jeisson Andrés Vergara Vargas
Sistema	NebulaStack — plataforma SaaS B2B multi-tenant de agendamiento y pagos
Repositorio Git	https://github.com/JRCR15/NebulaStack.git

1. Identificación del sistema

1.1. Dominio

NebulaStack es una plataforma SaaS **B2B multi-tenant** orientada a negocios que administran cupos finitos (salud, estética, diagnóstico automotriz, consultoría). Integra agenda centralizada, reserva bajo concurrencia, catálogo parametrizable por industria y orquestación de cobro con notificación transaccional.

La definición funcional se limita a capacidades de negocio observables. Las decisiones de arquitectura y construcción se desarrollan en las secciones siguientes.

1.2. Alcance del prototipo

La arquitectura de producto (diseño consolidado) contempla, a largo plazo, capacidades ampliadas (atención conversacional asistida por IA, analítica operacional y recuperación multi-región). El prototipo materializado concentra el valor demostrable en el flujo crítico:

autenticar → retener cupo (`HOLD`) → cobrar (Saga) → notificar (`CONFIRMED`), con compensación a `CANCELLED` ante fallo de cobro o ruptura del circuito.

Dos tenants de referencia operan en el prototipo: `cda` (diagnóstico automotriz) y `odonto` (salud dental), ambos en COP.

1.3. Objetivos

Objetivo general. Materializar en un prototipo ejecutable, con infraestructura como código, las estructuras, escenarios de calidad prioritarios y componentes FaaS del diseño arquitectónico, sin dispersar el alcance más allá del flujo crítico descrito.

Objetivos específicos.

1. Consolidar el diseño estructural (vistas C&C y despliegue) y los escenarios de calidad aplicables al prototipo.
2. Formalizar el diseño detallado: contratos de API y eventos, persistencia políglota con RLS, y máquina de estados de la cita.
3. Integrar dependencias externas mediante adaptadores controlados (pasarela y mensajería), preservando el dominio real en stores y broker.
4. Implementar al menos dos componentes bajo paradigma FaaS: `payment-orchestrator` y `notification-service`.
5. Definir empaquetado inmutable y estrategia de despliegue blue-green, con IaC declarativa para el mapeo a la nube.
6. Exponer el razonamiento diseño → construcción, los trade-offs y los límites deliberados del alcance actual.

---

2. Diseño arquitectónico consolidado

2.1. Vista de componentes y conectores — diseño objetivo

Las Figuras 1 y 2 corresponden a la **arquitectura objetivo** del producto (consolidación del diseño estructural y de tácticas). Describen el sistema completo, incluidos elementos aún no materializados en el runtime del prototipo (agente IA, analítica, IdP productivo, canales Meta, DR multi-región).

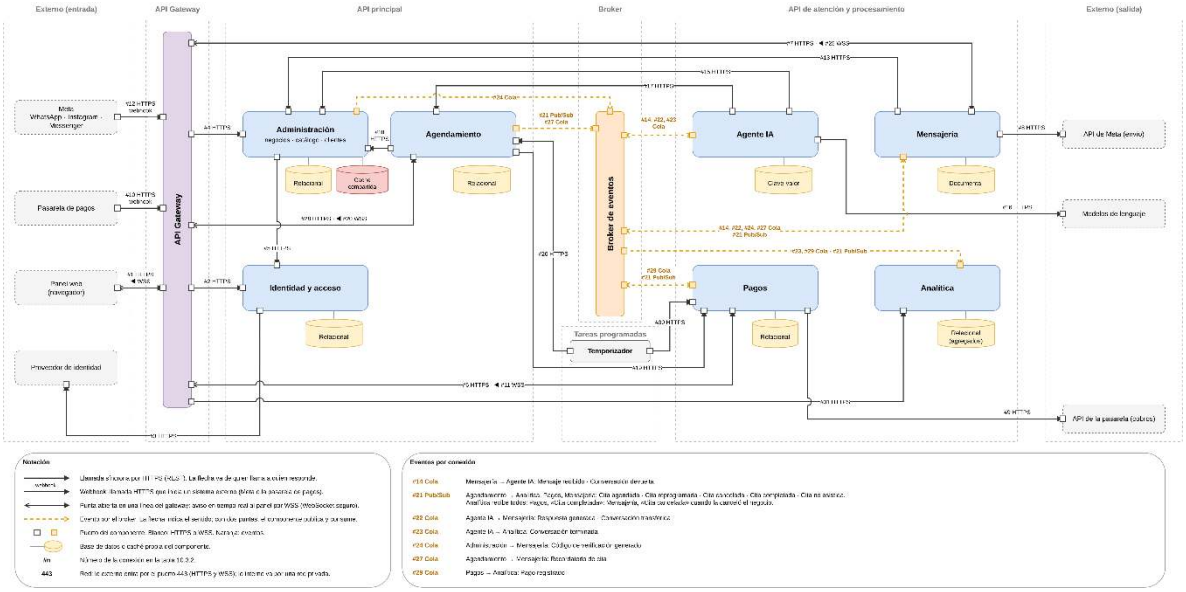


Figura 1. Vista de componentes y conectores (arquitectura objetivo).

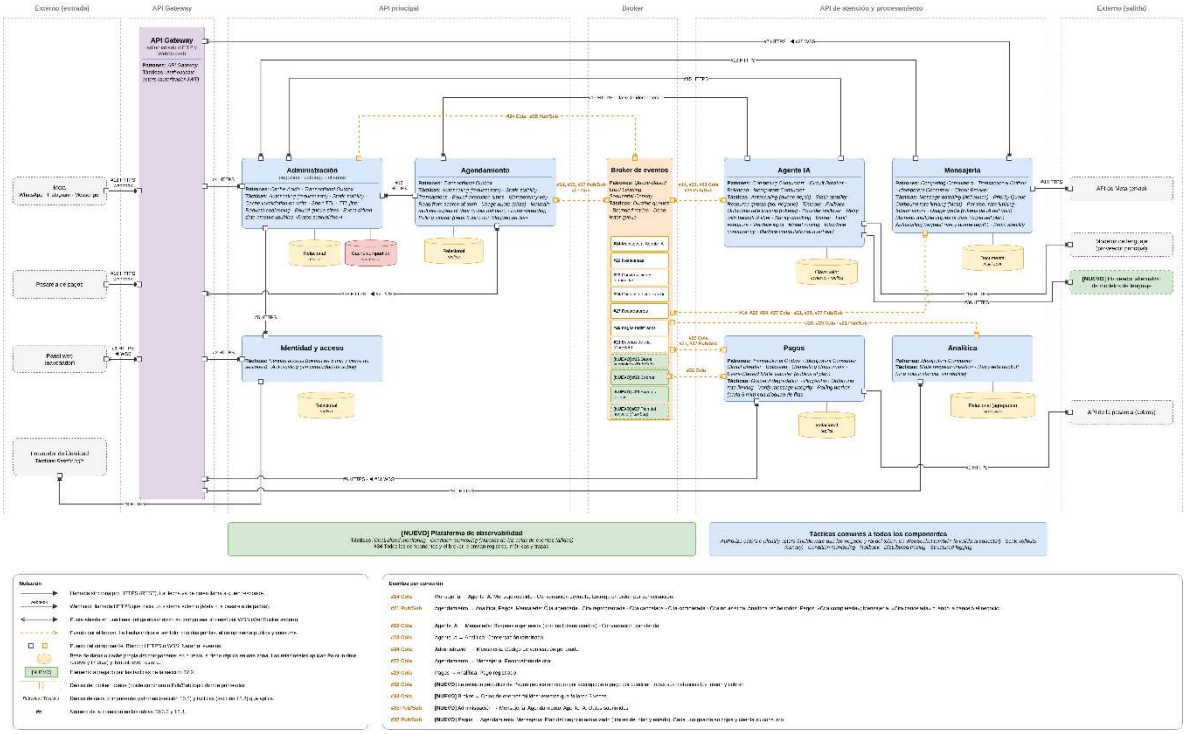


Figura 2. Componentes, conectores y tácticas en la arquitectura objetivo.

2.2. Vista de despliegue — diseño objetivo

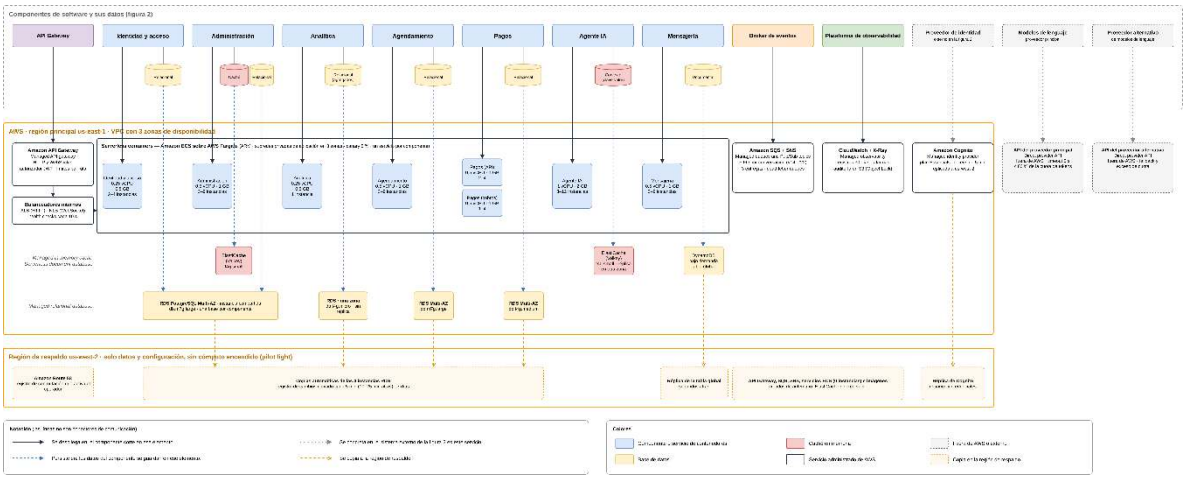


Figura 3. Mapeo a AWS (región primaria y sitio de recuperación). Arquitectura objetivo de despliegue en la nube.

El prototipo ejecutable se despliega en contenedores con plano de datos compartido y, para la estrategia de liberación, con flotas de aplicación blue-green detrás de un edge de enrutamiento. La infraestructura AWS se expresa en Terraform como modelo declarativo alineado a esta vista.

2.3. Vista C&C del prototipo (Figura 4) — patrones visibles

La Figura 1 **no etiqueta** el patrón Saga: es una vista de producto. La **Figura 4** es la vista C&C del *slice* materializado, donde **Saga, Transactional Outbox, Circuit Breaker y el lock Redis** aparecen como conectores explícitos. Así se evita la inconsistencia “usar la imagen de diseño y nombrar un patrón que no se ve en ella”.

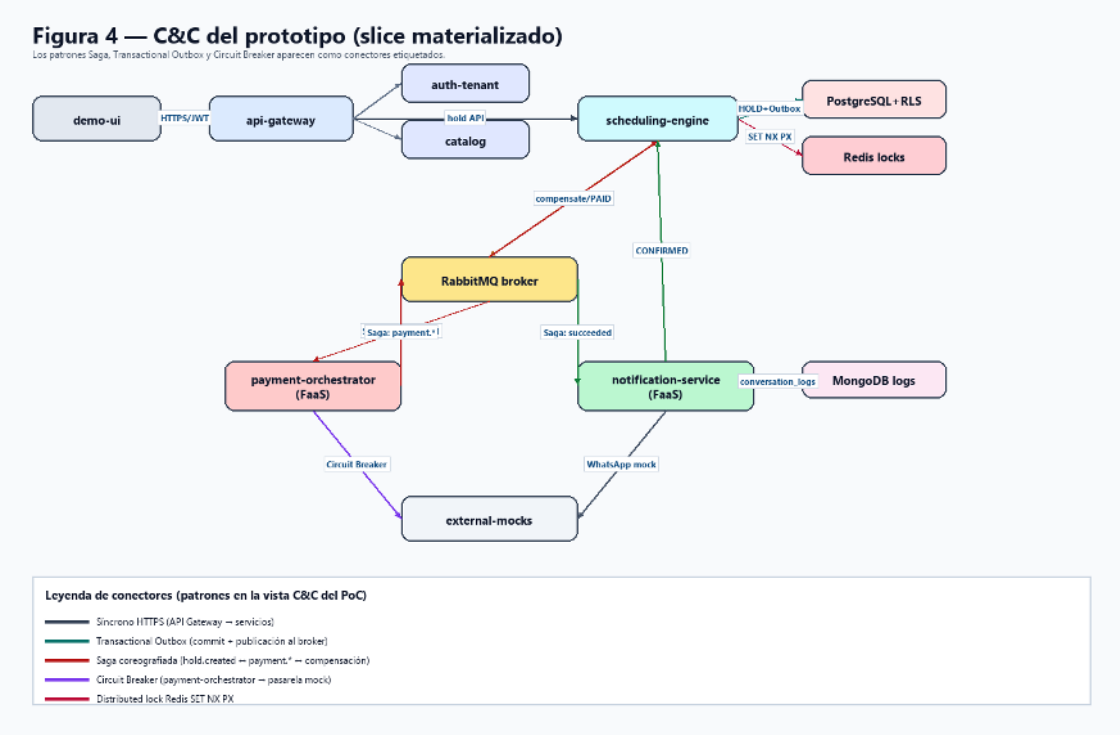


Figura 4. Vista de componentes y conectores del prototipo. Los patrones de calidad del flujo crítico están etiquetados en los conectores.

Patrón	Dónde en la Figura 4	Evidencia en runtime
API Gateway	`demo-ui` → `api-gateway` → servicios	JWT, overwrite de tenant, rate limit
Distributed lock (`SET NX PX`)	`scheduling-engine` → Redis	Un `201` y un `409` en carrera de cupo
Transactional Outbox	Scheduling → PostgreSQL + publicación a RabbitMQ	Outbox `processed = true`
Saga coreografiada	Broker ↔ `payment-orchestrator` ↔ Scheduling / Notification	`payment.succeeded` / `payment.failed`; sonda `666000`
Circuit Breaker	Payment FaaS → `external-mocks`	Timeout ~3 s; sonda `999000`
FaaS ×2	`payment-orchestrator`, `notification-service`	Workers locales; modelo Lambda + cola
RLS	PostgreSQL	Aislamiento `cda` ≠ `odonto`

2.4. Correspondencia diseño → prototipo

Capacidad	Estado en el prototipo	Ruta en el repositorio

API Gateway	Implementado — TypeScript / Fastify	`services/api-gateway`
Identidad y tenants	Implementado — JWT de prototipo (sin Cognito)	`services/auth-tenant-service`
Catálogo	Implementado — PostgreSQL	`services/catalog-service`
Agendamiento, Outbox y FSM	Implementado — Python / FastAPI	`services/scheduling-engine`
Pagos (Saga y Circuit Breaker)	Implementado — FaaS TypeScript	`services/payment-orchestrator`
Notificaciones	Implementado — FaaS TypeScript	`services/notification-service`
Adaptadores externos	Implementado	`services/external-mocks`
Broker de eventos	RabbitMQ local (CloudEvents); modelo AWS = SQS	Compose / `infra/terraform`
PostgreSQL con RLS	Implementado	`contracts/sql/`
Redis (locks)	Implementado	Compose
MongoDB (`conversation_logs`)	Implementado	notification-service
Blue-green	Edge + flotas blue/green	`deploy/scripts/bg-*.ps1`
IaC	Terraform (validate; sin apply de pago obligatorio)	`infra/terraform`
Agente IA / analítica / IdP / DR	Diseño objetivo (Fig. 1–3); no en runtime	—

2.5. Análisis de trade-offs

A continuación, se documentan dos tensiones centrales del prototipo (alineadas al diseño de calidad y a los ADR del repositorio).

TO-1 — Saga coreografiada + Outbox frente a 2PC (ADR-001)

Elemento	Contenido
Tensión	Consistencia fuerte inmediata del cobro vs disponibilidad y desacoplamiento bajo fallos de pasarela
Decisión	Saga coreografiada sobre eventos (`hold.created` → cobro → `payment.succeeded`
Alternativa descartada	Transacción distribuida 2PC entre scheduling y pasarela

Se gana	Resiliencia ante latencia/errores externos; compensación explícita a `CANCELLED`; conectores visibles en Fig. 4
Se cede	Consistencia eventual entre cupo retenido y estado de pago durante la ventana de la saga
Resolución	ADR-001 en `docs/adrs/ADR-001-saga-vs-2pc.md`
Ancla en el PoC	Sonda `666000` → compensación; Outbox evita eventos huérfanos

TO-2 — Base compartida + RLS frente a base por tenant (ADR-002)

Elemento	Contenido
Tensión	Aislamiento estricto de datos vs costo operativo y velocidad de entrega del PoC multi-tenant
Decisión	PostgreSQL compartido con Row-Level Security y tenant forzado desde el JWT en el gateway
Alternativa descartada	Una base de datos física por negocio
Se gana	Menor superficie de operación; mismos contratos SQL; demo con `cda` y `odonto`
Se cede	Disciplina estricta de contexto de tenant (pools de conexión); riesgo de fuga si se omite el claim
Resolución	ADR-002 en `docs/adrs/ADR-002-shared-db-rls.md`
Ancla en el PoC	Gateway pisa `x-tenant-id`; políticas RLS en `contracts/sql/`

2.6. Registro de ADRs

ADR	Decisión	Trade-off principal
ADR-001	Saga coreografiada con Outbox (frente a 2PC)	Consistencia eventual a cambio de disponibilidad
ADR-002	Base compartida con RLS (frente a DB por tenant)	Menor costo operativo; mayor disciplina de contexto
ADR-003	Broker local AMQP / SQS en nube	Experiencia de desarrollo frente a integración FaaS nativa
ADR-004	TypeScript y Python en el pin de lenguajes	Velocidad de entrega frente a homogeneidad
ADR-005	Secretos fuera del control de versiones	Seguridad operativa

3. Escenarios de calidad

Se prioriza el subconjunto de escenarios **verificables** sobre el prototipo. No se atribuyen al runtime mediciones de laboratorio no ejecutadas (p. ej. elasticidad masiva en ECS, Chaos Mesh o cargas sintéticas no corridas). El diseño de calidad de producto contempla un conjunto más amplio; ver Anexo C.

ID	Atributo	Fuente	Estímulo	Ambiente	Artefacto	Respuesta	Medida
QAS-01	Consistencia (cupo)	Clientes concurrentes	Dos retenciones sobre el mismo slot	Alta concurrencia	Scheduling y Redis	Una retención admitida; la otra rechazada	201 y 409 SLOT_ALREADY_LOCKED
QAS-02	Confiabilidad (Saga)	Pasarela	Cobro rechazado (402)	Operación normal	Payment FaaS y Scheduling	Compensación	CANCELLED + unlock (sonda 666000)
QAS-03	Resiliencia (CB)	Pasarela lenta/504	Latencia > umbral	Fallo externo	Payment FaaS	Fallo controlado	Timeout ~3 s → CANCELLED (sonda 999000)
QAS-04	Seguridad (tenant)	Cliente	Acceso cruzado	Multi-tenant	Gateway + RLS	Aislamiento	cda ≠ odonto
QAS-05	Disponibilidad	Operación	Arranque	Prototipo	Contenedores	Listos	/healthz
QAS-06	Observabilidad	Operación	hold→pago→aviso	Prototipo	Outbox, Mongo, UI	Trazabilidad	Outbox procesado; Mongo SENT si CONFIRMED

3.1. Tácticas aplicadas (mapeo a atributo)

Táctica o patrón	Atributo que soporta	Componente / ruta	Efecto observable
API Gateway	Seguridad, disponibilidad de borde	`services/api-gateway`	JWT, tenant forzado, rate limit
Distributed lock	Consistencia de cupo	`services/scheduling-engine` + Redis	Retención atómica TTL 120 s
Transactional Outbox	Confiabilidad de integración	scheduling-engine	Evento tras commit
Saga coreografiada	Confiabilidad / compensación	payment ↔ scheduling (Fig. 4)	`payment.succeeded` / `failed`
Circuit Breaker	Resiliencia	`services/payment-orchestrator`	Contención ante latencia/error
RLS por tenant	Seguridad	PostgreSQL	Aislamiento de filas
FaaS ×2	Elasticidad de workers (diseño)	payment + notification	Procesamiento asíncrono

---

4. Diseño detallado

4.1. Contratos

Tipo	Ubicación	Contenido
OpenAPI	`contracts/openapi/`	auth, catalog, scheduling, gateway-routes
AsyncAPI	`contracts/asyncapi/broker.yaml`	Eventos del broker
SQL y seeds	`contracts/sql/`, `contracts/seeds/`	Esquema, RLS, tenants, probes

CloudEvents	outbox / Rabbit	Tipos `nebula.*`, at-least-once

Las mutaciones exigen Bearer e `Idempotency-Key` en la retención. El tenant se deriva del token; el gateway no confía en un `x-tenant-id` del cliente.

4.2. Persistencia políglota

Almacén	Uso
PostgreSQL	Tenants, catálogo, citas, outbox; RLS
Redis	Lock de cupo y rate limit
MongoDB	`conversation_logs` (avisos)
Broker AMQP	CloudEvents entre productores y FaaS

4.3. Máquina de estados de la cita

```
CREATED → HOLD → PAID → CONFIRMED
                ↘ EXPIRED
HOLD → CANCELLED
```

Estado	Significado
`HOLD`	Cupo retenido; pendiente de cobro
`PAID`	Cobro exitoso en la saga
`CONFIRMED`	Notificación despachada
`CANCELLED`	Compensación o cancelación explícita
`EXPIRED`	Vencimiento del hold sin cobro exitoso

4.4. Retención atómica

Redis `SET … NX PX` (TTL 120 000 ms) en el mismo flujo que persiste `HOLD` y el registro de outbox, eliminando la carrera *check-then-act*.

---

5. Construcción del prototipo

5.1. Componentes materializados

1. **scheduling-engine** (`services/scheduling-engine`) — Redis, FSM, Outbox.
2. **payment-orchestrator** (`services/payment-orchestrator`) — FaaS #1, Saga, Circuit Breaker (opossum).
3. **notification-service** (`services/notification-service`) — FaaS #2, Mongo, WhatsApp mock → `CONFIRMED`.
4. **api-gateway**, **auth-tenant-service**, **catalog-service** — frontera y catálogo.
5. **external-mocks** — pasarela y mensajería.
6. **demo-ui** (`deploy/demo`) — recorrido funcional y sondas.

5.2. Integraciones externas

Solo pasarela y WhatsApp se resuelven con adaptadores. El dominio (hold, outbox, saga, FSM, RLS, logs) es real.

Condición	Adaptador	Resultado
Precio nominal	Cobro OK	`CONFIRMED` + Mongo `SENT`
Sonda `666000`	`402`	Saga → `CANCELLED`
Sonda `999000`	Delay / `504`	Circuit Breaker → `CANCELLED`

5.3. Prácticas nativas de la nube

Práctica	Aplicación en el prototipo
Statelessness	Sin estado de sesión en proceso; estado en Postgres/Redis/Mongo
Health checks	Endpoint `/healthz` (un nivel) en servicios de borde y aplicación
Graceful shutdown	**Parcial:** los contenedores responden a señales de orquestación Docker; no se afirma un draining formal idéntico (p. ej. 15 s) en todos los servicios del PoC
Structured logging	Logs a stdout de contenedor; **no** se declara cobertura total de TraceID W3C al 100 %
Contenedores OCI	Compose + imágenes etiquetables por commit
k6 / Chaos Mesh	**No ejecutados** — sin métricas inventadas

5.4. Functions-as-a-Service y cold start

#	Componente	Lenguaje	Local	Modelo AWS
1	`payment-orchestrator`	TypeScript	Contenedor + consumer Rabbit	Lambda + SQS
2	`notification-service`	TypeScript	Contenedor + consumer Rabbit	Lambda + SQS

Cold start y límites de ejecución. En el prototipo Compose los workers permanecen calientes (sin cold start de Lambda). En el diseño cloud, el mismo handler se empaqueta para Lambda: inicialización liviana, timeout acotado y sin dependencia de Provisioned Concurrency en esta entrega. El segundo FaaS es **notificación**, no el agente IA.

5.5. Contenedores del prototipo

Contenedor	Tecnología	Responsabilidad
api-gateway	TS / Fastify	JWT, tenant, rate limit
auth-tenant-service	TS	Tokens demo
catalog-service	TS	Precios / servicios
scheduling-engine	Python FastAPI	Hold, FSM, outbox
payment-orchestrator	TS FaaS	Saga + CB
notification-service	TS FaaS	Aviso + CONFIRMED
external-mocks	TS	Pasarela / WhatsApp
postgres / redis / mongo / rabbitmq	—	Stores y broker
demo-ui	nginx + HTML/JS	Demo

---

6. Despliegue e infraestructura como código

6.1. Entorno de prototipo

Configuración	Propósito
Stack unificado (`docker-compose.yml`)	Desarrollo y E2E
Blue-green (`bg-up` / `bg-deploy` / `bg-switch`)	Cutover y rollback

Acceso del prototipo: UI `http://localhost:3000`, API `http://localhost:8080`. **No hay URL pública cloud** en esta entrega; el sistema evaluable es local vía Compose o edge blue-green.

6.2. Estrategia blue-green

- Plano de datos compartido; flotas blue/green; edge conmuta el color activo.
- Artefactos inmutables: `IMAGE_TAG=$(git rev-parse --short HEAD)`.
- Rollback: `bg-switch` al color anterior.

6.3. Secuenciación de cambios de esquema / contrato frente al despliegue

Cuando aplica un cambio de contrato o esquema:

1. **Expand:** agregar columnas/campos nullable o versiones compatibles en OpenAPI/AsyncAPI y SQL; desplegar escritores que llenan ambas formas.
2. **Migrate / backfill:** lectores adoptan el nuevo contrato; verificar con E2E.
3. **Contract:** retirar lo obsoleto en un despliegue posterior.

El cutover **blue-green** cambia el tráfico de aplicación sin alterar el plano de datos compartido; por eso las migraciones expand-contract deben completarse (o ser

compatibles hacia atrás) **antes** de conmutar el edge a la flota nueva. Un cambio incompatible de esquema no se “arregla” solo con blue-green.

6.4. Infraestructura como código

Herramienta: **Terraform** (`infra/terraform`). Provisiona (diseño): red, ECR, colas SQS, datos opcionales, Lambda FaaS, esqueleto ECS. Secretos vía `TF_VAR_*` (no en git). Evidencia classroom: `infra/terraform/validate.ps1` (`init -backend=false` + `validate`). No se exige `apply` completo de pago (NAT/RDS/Fargate generan costo).

6.5. Empaquetado y repositorio

Imágenes OCI etiquetadas por commit. Handlers FaaS compartidos entre contenedor local y modelo Lambda. Repositorio: https://github.com/JRCR15/NebulaStack.git

---

7. Discusión y reflexión (design-to-construction)

7.1. Design-to-Construction Rationale

El diseño estructural y de calidad (vistas objetivo Fig. 1–3) fijó Gateway, broker, pagos asíncronos y multi-tenant. La construcción reveló un **gap de representación**: la Saga del PoC no estaba etiquetada en la C&C de producto; se resolvió con la **Figura 4** y la tabla patrón→conector. Otro hallazgo: la visión conversacional/IA del diseño amplio no cabe en el slice E2E; se delimitó explícitamente como roadmap, no como runtime.

7.2. Implementation Challenges

1. Contexto RLS limpio en pools de conexión.
2. TTL Redis 120 s frente a latencia de cobro.
3. Coexistencia de consumidores blue/green sobre el mismo broker (aceptable para demo de cutover HTTP).
4. Mantener honestidad documental (sin métricas no medidas).

7.3. Quality Attribute Verification

Escenario	Cómo se verificó	¿Cumple la medida del PoC?
QAS-01 cupo	Carrera de holds / UI	Sí — `201/409`
QAS-02 Saga	Sonda `666000` + E2E + BD	Sí — `CANCELLED` + unlock
QAS-03 CB	Sonda `999000` + E2E	Sí — fallo controlado ~3 s
QAS-04 tenant	Login CDA vs Odonto + RLS	Sí — catálogos/citas aislados

No se afirma verificación runtime de elasticidad multi-AZ, Chaos Mesh ni los QAS ampliados del diseño de producto (Anexo C).

7.4. Cloud-Native Evaluation

El sistema en ejecución es **cloud-native en el sentido del PoC**: cómputo sin estado de sesión, contenedores, empaquetado inmutable, IaC declarativa, FaaS (handlers), despliegue progresivo blue-green, persistencia administrada vía Compose. **No** es un despliegue productivo multi-región ni con IdP/pasarelas reales; esa distancia se declara a propósito.

7.5. Lessons Learned

1. Idempotencia e IDs estables son obligatorios con at-least-once.
2. Pocos QAS con evidencia superan un catálogo amplio sin corrida.
3. Separar stack E2E de blue-green evita interferencias.
4. Los patrones deben verse en la vista C&C del *slice* que se demuestra, no solo en prosa.

---

8. Verificación realizada

8.1. Recorrido funcional

Auth → hold → pago → confirmación para `cda` y `odonto`; compensación y Circuit Breaker vía sondas; UI en `:3000`.

8.2. E2E automatizado

Scripts `deploy/scripts/e2e-hold-pay-notify.ps1` (+ asserts Postgres/Mongo/Redis).

8.3. Blue-green

`bg-up` → `bg-deploy` → `bg-switch` (rollback).

8.4. IaC

`.\infra\terraform\validate.ps1` — configuración válida sin apply de pago.

8.5. Credenciales de referencia

Contexto	Identidad	Secreto
UI CDA	`admin@cda.example`	`Demo123!`
UI Odonto	`admin@odonto.example`	`Demo123!`
E2E client_credentials	`demo-client`	`demo-secret`

8.6. Puntos de acceso

Recurso	Dirección
Interfaz demo	http://localhost:3000
API Gateway	http://localhost:8080
RabbitMQ UI	http://localhost:15672 (`guest/guest`)
Repositorio	https://github.com/JRCR15/NebulaStack.git
URL cloud pública	No aplica en esta entrega (prototipo local)

---

9. Conclusiones

NebulaStack se entrega como **prototipo ejecutable** de construcción continua: multi-tenant, lock de cupo, Outbox, **Saga visible en la Figura 4**, Circuit Breaker, FaaS ×2, blue-green e IaC Terraform alineada al diseño en nube.

Las Figuras 1–3 fijan la arquitectura objetivo; la Figura 4 y los trade-offs TO-1/TO-2 anclan el diseño al runtime demostrable.

Anexo A — Planos del sistema

Plano	Contenido
Diseño objetivo	Fig. 1–3: C&C, tácticas, despliegue AWS, visión ampliada
Prototipo actual	Fig. 4: Saga/Outbox/CB; Compose; E2E; blue-green; Terraform validate
Evolución prevista	Integraciones productivas, agente IA, DR, consumidores por slot

Anexo B — Glosario

Término	Definición
HOLD	Cupo retenido con lock Redis
Outbox	Publicación fiable tras commit
Saga	Coreografía de cobro con compensación
Circuit Breaker	Contención ante latencia/error de pasarela
RLS	Seguridad a nivel de fila por tenant
Blue-green	Dos flotas + edge de conmutación

Anexo C — Escenarios de diseño no verificados en runtime

El diseño de calidad de producto formaliza un conjunto amplio de QAS (elasticidad agresiva, failover multi-AZ/región, proveedor de IA, analítica, etc.). **Este entregable no los afirma como medidos en el PoC.** Quedan como backlog de verificación cuando exista entorno cloud instrumentado. El subconjunto QAS-01…06 de la sección 3 es el contrato de evidencia de Delivery 4.